Name: Aaron Ludwig A. Altar **Activity #1: Plotting Sequence**
Section: CPE4B **Date: Aug. 26, 2026**

Digital Signal Processing (DSP): Assessment

Part 1: Signal Classification

Determine whether each of the following real-world signals is **Continuous-Time (Analog)** or **Discrete-Time (Digital)**, and explain your reasoning in one sentence.

1. The temperature recorded continuously by a liquid thermometer.
   - **Continuous-Time (Analog),** the Mercury/alcohol column expands smoothly and continuously with temperature, existing at every instant in time, not just at sampled points.

2. The daily closing price of a stock on the stock market.
   - **Discrete-Time (Digital),** the price is only recorded once per trading day, so it exists as a sequence of values indexed by discrete day-numbers (n), with nothing defined "between" days.

3. Audio played directly through an acoustic instrument.
   - **Continuous-Time (Analog),** the sound wave (air pressure variation) travels and varies continuously throughout time and space before it ever reaches a microphone or sensor.
4. An MP3 file stored on a smartphone.

   - **Discrete-Time (Digital),** the audio has been sampled and quantized into a finite sequence of numeric values stored as bits, existing only at discrete sample indices.

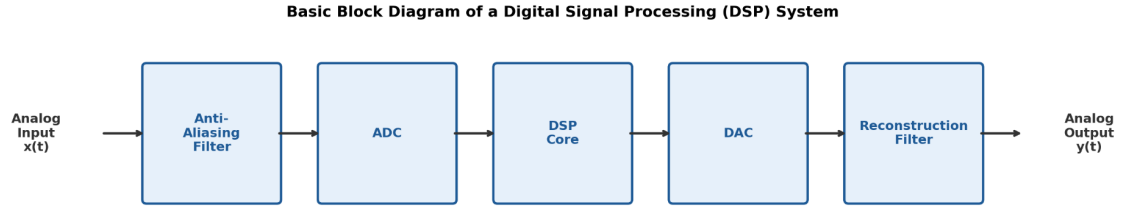
Part 2: Core Concepts (Short Answer)

1. **Sampling:** Briefly define sampling and state the Nyquist Sampling Theorem. What happens if the sampling frequency (f_s) is less than twice the highest frequency present in the signal ($2f_{max}$)?
   - **Sampling** is the process of converting a continuous-time signal into a discrete-time signal by recording its amplitude at uniform time intervals (T), called the sampling period, with sampling frequency fs = 1/T. The Nyquist Sampling Theorem states that a signal can be completely and accurately reconstructed from its samples only if it is sampled at a rate at least twice its highest frequency component (fs ≥ 2fmax), a minimum known as the Nyquist rate. If fs is less than 2fmax, a problem called aliasing occurs, higher frequencies fold back and get misrepresented as false, lower frequencies in the sampled data, causing the reconstructed signal to come out distorted and unrecognizable compared to the original. This loss of information is permanent; no amount of processing afterward can undo it once it happens during sampling. This is precisely why real DSP systems use an anti-aliasing (low-pass) filter before the ADC, to block out frequencies above fs/2 and prevent aliasing from

occurring in the first place.

2. **Block Diagram:** Draw or describe the basic block diagram for a Digital Signal Processing system processing a real-world analog input signal (include ADC, DSP core, and DAC).

- In summary, a DSP system takes a real-world **analog input x(t)** and passes it through an **anti-aliasing filter**, which removes frequencies above fs/2 to prevent aliasing. It then goes through the **ADC**, which samples and quantizes it into digital sequence x[n]. This is followed by the **DSP core**, which processes it through filtering, FFT, or compression to produce y[n]. Next, the **DAC** converts y[n] back into a staircase-shaped analog signal. Finally, a **reconstruction filter** smooths it into a clean signal, resulting in the final **analog output y(t)**.



Part 3: Basic Signal Operations

Given the discrete-time sequence **x[n] = {2, -1, 3, 0, 4}** where the origin **n = 0** is at value **2.**
Create a python program and:

1. Sketch or list the values for $\mathbf{y_1[n] = x[n - 2]}$ (Time Shift).
2. Sketch or list the values for $\mathbf{y_2[n] = x[-n]}$ (Time Reversal).
3. Compute the total energy **E** of the sequence **x[n]** using the formula:

$E = \Sigma |x[n]|^2$

```
# PART 3: Basic Signal Operations
# x[n] = {2, -1, 3, 0, 4}, n=0 at value 2

import matplotlib.pyplot as plt

x = {0: 2, 1: -1, 2: 3, 3: 0, 4: 4}  # x[n] as {index: value}

def show(seq, name):
    # prints n and value pairs, sorted by n
    for n in sorted(seq.keys()):
        print(f"  n = {n} -> {seq[n]}")
    print()

print("x[n] (original):")
```

Name: Aaron Ludwig A. Altar **Activity #1: Plotting Sequence**

Section: CPE4B **Date: Aug. 26, 2026**

```
show(x, "x[n]")

# 1. Time Shift: y1[n] = x[n-2] -> shift right by 2
y1 = {n + 2: val for n, val in x.items()}
print("y1[n] = x[n-2]:")
show(y1, "y1")

# 2. Time Reversal: y2[n] = x[-n] -> mirror around n=0
y2 = {-n: val for n, val in x.items()}
print("y2[n] = x[-n]:")
show(y2, "y2")

# 3. Energy: E = sum |x[n]|^2
E = sum(abs(val) ** 2 for val in x.values())
print(f"Energy E = {E}")

y2 = {-n: val for n, val in x.items()}
print("y2[n] = x[-n]:")
show(y2, "y2")

# 3. Energy: E = sum |x[n]|^2
E = sum(abs(val) ** 2 for val in x.values())
print(f"Energy E = {E}")

# Plot all three as stem diagrams
def stem_plot(ax, seq, title):
    ns = sorted(seq.keys())
    vals = [seq[n] for n in ns]
    ax.stem(ns, vals, basefmt=" ")
    ax.axhline(0, color="black", linewidth=0.8)
    ax.set_title(title)
    ax.set_xlabel("n")
    ax.set_ylabel("Amplitude")
    ax.set_xticks(range(min(ns) - 1, max(ns) + 2))
    ax.grid(True, linestyle="--", alpha=0.5)
```

Name: Aaron Ludwig A. Altar **Activity #1: Plotting Sequence**

Section: CPE4B **Date: Aug. 26, 2026**

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
stem_plot(axes[0], x,  "x[n]")
stem_plot(axes[1], y1, "y1[n] = x[n-2]")
stem_plot(axes[2], y2, "y2[n] = x[-n]")
fig.suptitle(f"Energy E = {E}")
plt.tight_layout()
plt.savefig("dsp_part3_diagram.png", dpi=150)
plt.show()
```

```
    x[n] (original):
      n = 0 -> 2
      n = 1 -> -1
      n = 2 -> 3
      n = 3 -> 0
      n = 4 -> 4

    y1[n] = x[n-2]:
      n = 2 -> 2
      n = 3 -> -1
      n = 4 -> 3
      n = 5 -> 0
      n = 6 -> 4

    y2[n] = x[-n]:
      n = -4 -> 4
      n = -3 -> 0
      n = -2 -> 3
      n = -1 -> -1
      n = 0 -> 2

    Energy E = 30
```

```
# Plot all three as stem diagrams

def stem_plot(ax, seq, title):

  ns = sorted(seq.keys())

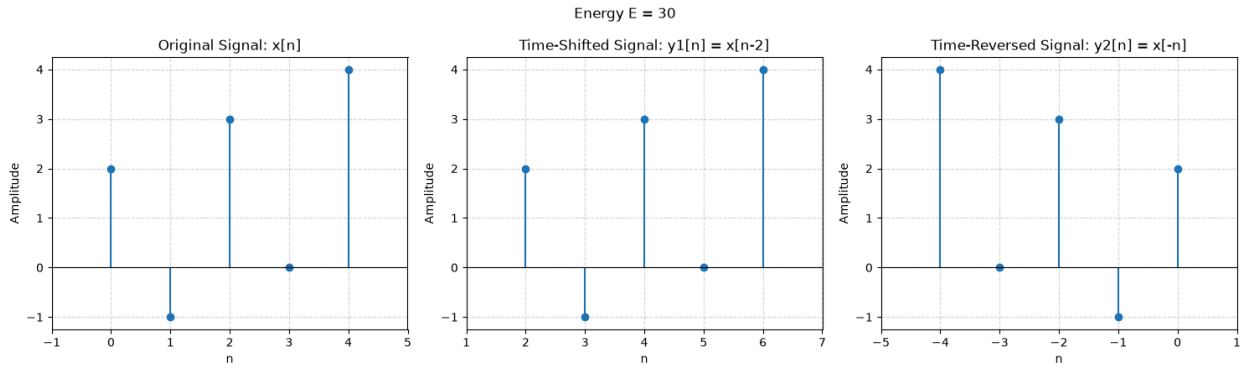
  vals = [seq[n] for n in ns]
```

Name: Aaron Ludwig A. Altar **Activity #1: Plotting Sequence**

Section: CPE4B **Date: Aug. 26, 2026**

```
    ax.stem(ns, vals, basefmt=" ")
    ax.axhline(0, color="black", linewidth=0.8)
    ax.set_title(title)
    ax.set_xlabel("n")
    ax.set_ylabel("Amplitude")
    ax.set_xticks(range(min(ns) - 1, max(ns) + 2))
    ax.grid(True, linestyle="--", alpha=0.5)


fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
stem_plot(axes[0], x,  "Original Signal: x[n]")
stem_plot(axes[1], y1, "Time-Shifted Signal: y1[n] = x[n-2]")
stem_plot(axes[2], y2, "Time-Reversed Signal: y2[n] = x[-n]")
fig.suptitle(f"Energy E = {E}")
plt.tight_layout()
plt.savefig("dsp_part3_diagram.png", dpi=150)
plt.show()
```



Name: Aaron Ludwig A. Altar Activity #1: Plotting Sequence

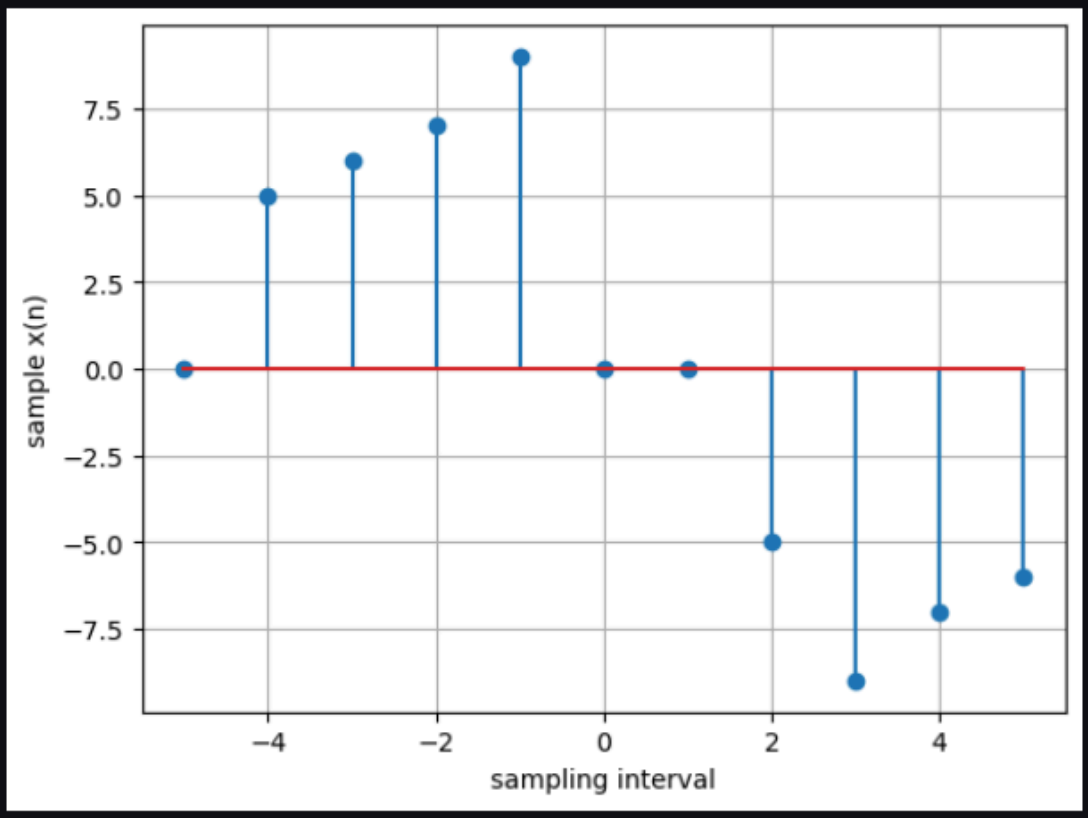
Section: CPE4B Date: Aug. 26, 2026

Additional Graphs

```
[4]
n = np.arange(-5,6)
x = np.array([0, 5, 6 ,7,9,0,0, -5, -9, -7, -6])
print(n)
print(x)
```

```
[-5 -4 -3 -2 -1  0  1  2  3  4  5]
[ 0  5  6  7  9  0  0 -5 -9 -7 -6]
```

```
[5]
plt.stem(n,x)
plt.xlabel('sampling interval')
plt.ylabel('sample x(n)')
plt.grid(True)
plt.show()
```

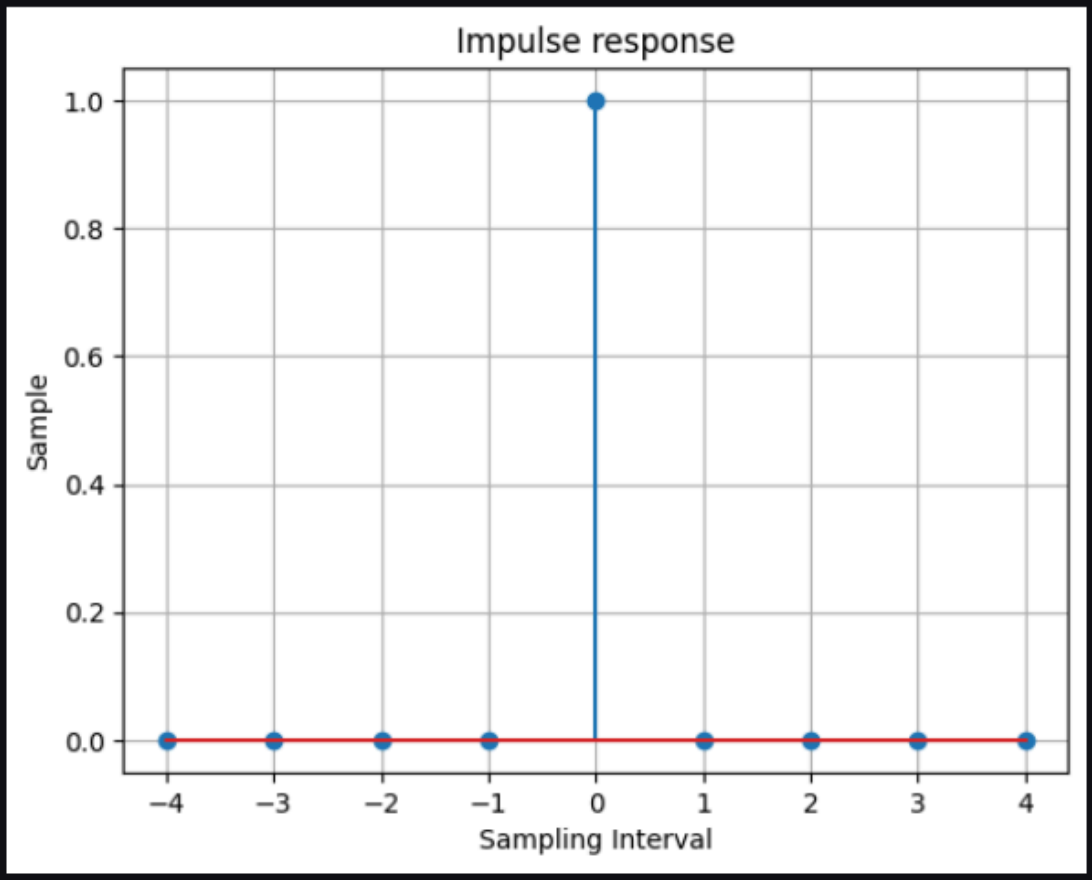


Name: Aaron Ludwig A. Altar **Activity #1: Plotting Sequence**

Section: CPE4B **Date: Aug. 26, 2026**

```
n = np.array([-4, -3, -2, -1, 0, 1, 2, 3, 4])
xn = ([ 0, 0, 0, 0, 1, 0, 0, 0, 0])
plt.stem(n,xn)
plt.title('Impulse response')
plt.xlabel('Sampling Interval')
plt.ylabel('Sample')
plt.grid(True)
plt.show()
```

[6]

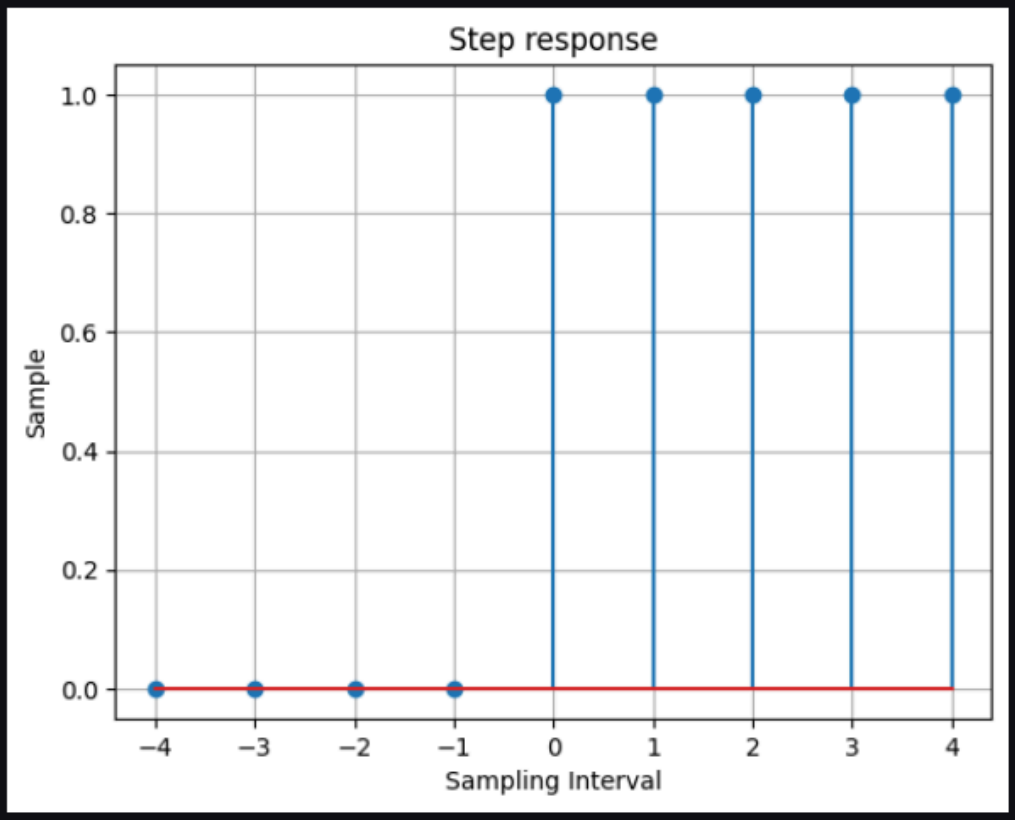


Name: Aaron Ludwig A. Altar **Activity #1: Plotting Sequence**

Section: CPE4B **Date: Aug. 26, 2026**

```
n = np.array([-4, -3, -2, -1, 0, 1, 2, 3, 4])
xn = ([ 0, 0, 0, 0, 1, 1, 1, 1, 1])
plt.stem(n,xn)
plt.title('Step response')
plt.xlabel('Sampling Interval')
plt.ylabel('Sample')
plt.grid(True)
plt.show()
```

[7]

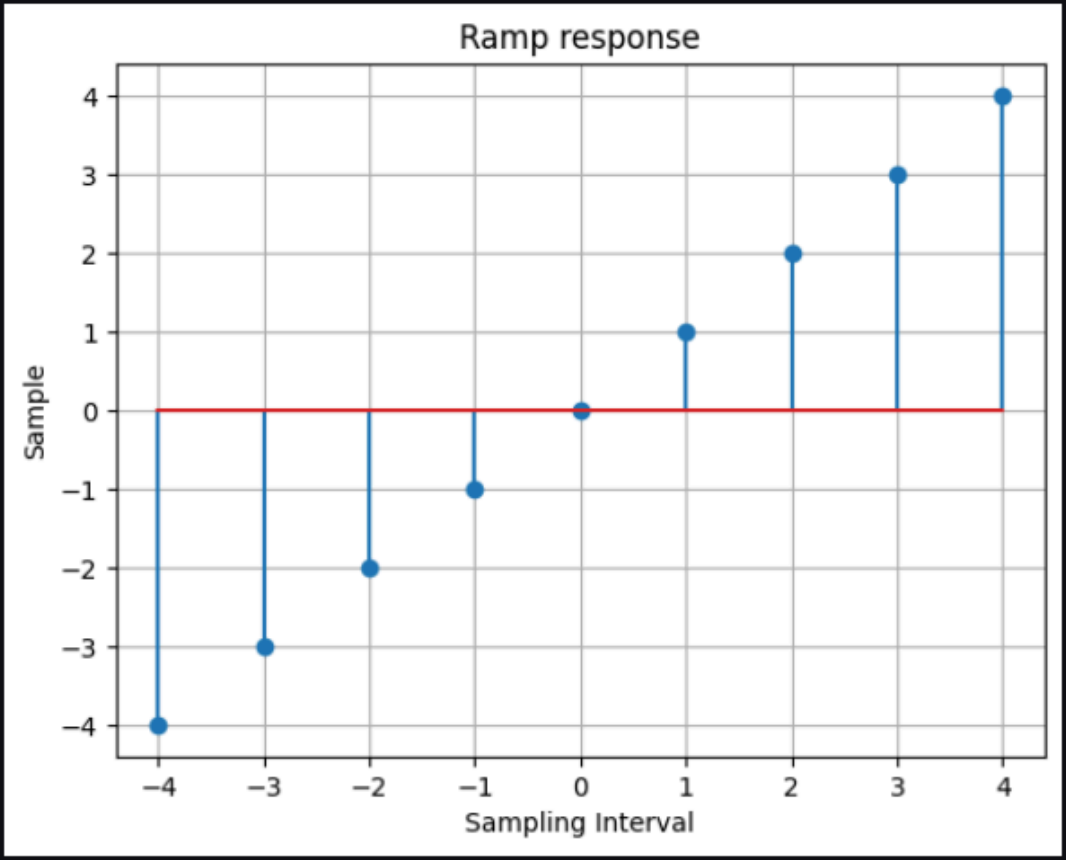


Name: Aaron Ludwig A. Altar Activity #1: Plotting Sequence

Section: CPE4B Date: Aug. 26, 2026

```
n = np.array([-4, -3, -2, -1, 0, 1, 2, 3, 4])
xn = ([-4, -3, -2, -1, 0, 1, 2, 3, 4])
plt.stem(n,xn)
plt.title('Ramp response')
plt.xlabel('Sampling Interval')
plt.ylabel('Sample')
plt.grid(True)
plt.show()
```

[8]



Name: Aaron Ludwig A. Altar **Activity #1: Plotting Sequence**

Section: CPE4B **Date: Aug. 26, 2026**

```
n = np.arange(-6.2,6.3,0.1) #1 and #2. n = np.arange(-3.1,3.2,0.1), #3 and #4 n = np.arange(-6.2,6.3,0.1)
xn = np.cos(n) #1 np.sin(n) #2 np.cos(n) #3 np.sin(n) #4 np.cos(n)
plt.stem(n,xn)
plt.title('Response Wave') #Sin wave/ Cosine wave
plt.xlabel('Sampling Interval')
plt.ylabel('Sample')
plt.grid(True)
plt.show()
```

[9]

